UNIT IV

OBJECT ORIENTED PROGRAMMING USING JAVA

TOPICS

Classes, objects and constructor in Java, Implementing inheritance and polymorphism - dynamic binding, method overriding, abstract classes and methods. Interfaces - Interfaces vs. Abstract classes, defining an interface, implementing interfaces, accessing implementations through interface references, extending interfaces.

Class

- A class is a user-defined reference type that describes the state and behavior common to objects of that type.
- It acts as a blueprint or template. State is represented by fields (variables), while behavior is represented by methods.
- A class may also contain constructors, nested types, and other members.

Class

- Synatx:

```
class ClassName {
        field declaration;
        method declaration;
}
```

- Example:

```
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println("Roll number: "
+ rollNo);
        System.out.println("Name: " +
name);
    }
}
```

Object

- An object is an instance of a class. It has identity, state, and behavior.
- State is stored in the object's instance fields; behavior is provided by its methods.
- In Java, a variable of a class type normally holds a reference to an object rather than the object's full contents directly.
- When you create an object using the new keyword, memory is allocated for the object, and the initial values for its properties are set.

Object

- Syntax

```
// 1. Declare a reference variable
                Student s1;
// 2. Create an object and assign the reference
                 s1 = new Student();
// 3. Combined form: Declare reference and create object
                Student s2 = new Student();
```

Object

```
// Define the Student class
class Student {

    // Instance variables
    int rollNo;
    String name;

    // Method to display student details
    void display() {
        System.out.println(rollNo + " - " + name);
    }
}
```

```
// Main class
class StudentDemo {
    public static void main(String[] args) {

        // Create an object of the Student class
        Student s1 = new Student();

        // Assign values to instance variables
        s1.rollNo = 101;
        s1.name = "Asha";

        // Call the display() method
        s1.display();
    }
}
```

Constructors in Java

- A constructor is a special member used to initialize a newly created object.
- Its name must match the class name, and it has no return type not even void.
- A constructor is invoked when an object is created with new.
- Constructors may accept parameters so that different objects can begin with different state.

Constructors in Java

- Constructors can be overloaded by using different parameter lists.
- Constructors are not inherited and cannot be overridden.
- If no constructor is declared, the compiler supplies a no-argument default constructor. Once any constructor is declared, that automatically supplied constructor is not provided.
- Constructors cannot be abstract or static.

Constructors in Java

No-Argument Constructor :

A no-argument constructor is a constructor that does not accept any parameters. It can be written explicitly to initialize an object with specific values.

Parameterized Constructor:

A parameterized constructor is a constructor that accepts one or more parameters to initialize an object's instance variables with specific values.

No-Argument Constructor

```
class Book {
    String title;

    // Explicitly defined no-argument constructor
    Book() {
        title = "Untitled";
    }

    // Method to display the book title
    void display() {
        System.out.println(title);
    }
}
```

```
// Main class
class BookDemo {
    public static void main(String[] args) {

        // Create a Book object
        Book b = new Book();

        // Call the display() method
        b.display();
    }
}
```

Parameterized Constructor

```
class Book {
    String title;
    double price;

    // Parameterized constructor
    Book(String t, double p) {
        title = t;
        price = p;
    }

    // Method to display book details
    void display() {
        System.out.println(title + ": Rs. " + price);
    }
}
```

```
// Main class
class BookDemo {
    public static void main(String[] args) {

        // Create objects using the parameterized
constructor
        Book b1 = new Book("Java Basics", 450.0);
        Book b2 = new Book("Data Structures", 520.0);

        // Display book details
        b1.display();
        b2.display();
    }
}
```

Constructor overloading

Constructor overloading means defining multiple constructors in the same class with different parameter lists. Each constructor can initialize an object in a different way.

Example:

```
class Rectangle {
   int length, breadth;
Rectangle() {// Constructor 1: No-argument constructor
      length = 1;
      breadth = 1;
   }
Rectangle(int side) {// Constructor 2: One-parameter constructor
      length = side;
      breadth = side;
   }
Rectangle(int length, int breadth) {// Constructor 3: Two-parameter constructor
      this.length = length;
      this.breadth = breadth;
   }
    int area() {            // Method to calculate area
      return length * breadth;
   }
}
class RectangleDemo {                // Main class
   public static void main(String[] args) {
      Rectangle r1 = new Rectangle(); // Calls the no-argument constructor
      Rectangle r2 = new Rectangle(5); //Calls the one-parameter constructor
      Rectangle r3 = new Rectangle(4, 6); //Calls the two-parameter constructor
      System.out.println("Area of r1: " + r1.area());
      System.out.println("Area of r2: " + r2.area());
      System.out.println("Area of r3: " + r3.area());
   }
}
```

Using this in Constructor

- this refers to the current object.
- It helps distinguish instance fields from parameters with the same names.

```
class Student {
    int rollNumber;
    String name;
   Student(int rollNumber, String name) {       // Parameterized constructor
        this.rollNumber = rollNumber;       // Initialize instance variables using this
        this.name = name;
    }
}
```

Inheritance

- In Java, Inheritance means creating new classes based on existing ones. A class that inherits from another class can reuse the methods and fields of that class. In addition, you can add new fields and methods to your current class as well.
- The class which inherits the properties of another is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class).

Inheritance

- It represents a parent-child relationship between two classes. This parent-child relationship is also known as an IS-A relationship.

Need of Inheritance

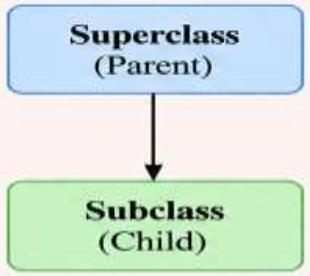
- Code Reusability
- Extensibility
- Implementation of Method Overriding:.
- Achieving Abstraction:

Inheritance

Types of Inheritance in Java

1. Single Inheritance

A class inherits from only one class.

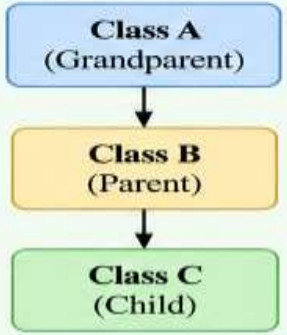


Key Points

- One parent class and one child class.
- Most commonly used type.
- The child class inherits the properties (fields and methods) of the parent class.

2. Multilevel Inheritance

A class inherits from a class, which is itself a subclass of another class.

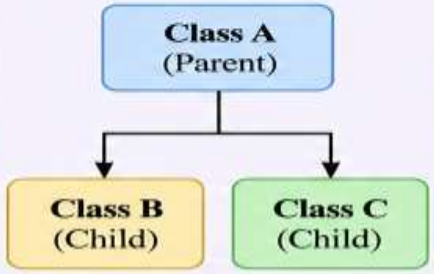


Key Points

- Inheritance in multiple levels.
- Forms a chain.
- Each class (except the topmost class) is both a subclass and a superclass.

3. Hierarchical Inheritance

Multiple classes inherit from the same parent class.

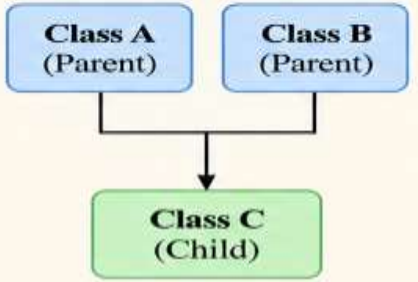


Key Points

- One parent class and multiple child classes.
- All child classes share the same parent class.
- The child classes inherit the properties of the parent class.

4. Multiple Inheritance

A class inherits from more than one class.

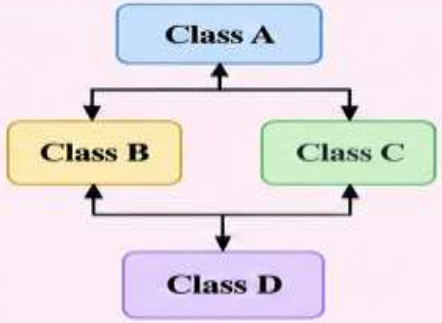


Key Points

- One class inherits from more than one parent class.
- Not supported directly in Java using classes.
- Can be achieved in Java using interfaces.
- Useful when a class needs to combine features from multiple sources.

5. Hybrid Inheritance

A combination of two or more types of inheritance.



Key Points

- Combination of two or more types of inheritance (single, multilevel, hierarchical).
- Not supported directly in Java using classes.
- Can be achieved in Java using interfaces.
- Useful for complex and flexible designs.

Inheritance

Defining a Subclass:

A subclass is defined as

```
class subclassname extends superclassname
{
        variable declaration;
        methods declaration;
}
```

Inheritance

- Keyword extends signifies that the properties of superclass are extended to the subclass.
- Subclass will now contain its own variables and methods as well those of superclass.

Example: single inheritance

```
class Room {
    int length;
    int breadth;
```

```
 Room(int x, int y) {
        length = x;
        breadth = y;
    }
int area() {
        return (length * breadth);
    }
}

// Inheriting Room
class BedRoom extends Room {
    int height;

    BedRoom(int x, int y, int z) {
        super(x, y);
        height = z;
    }
int volume() {
        return (length * breadth * height);
    }
}

class InherTest {
    public static void main(String args[]) {
        BedRoom room1 = new BedRoom(14, 12, 10);

        int area1 = room1.area();
        int volume1 = room1.volume();

        System.out.println("Area1 = " + area1);
        System.out.println("Volume1 = " + volume1);
    }
}
```

super Keyword

- super refers to the immediate superclass part of the current object.
- super.methodName() calls an accessible superclass method.
- super(...) calls a superclass constructor and must be the first statement in a constructor.
- If no constructor call is written, Java inserts super() when it is valid to do so.

super Keyword

```
class Person {
void display() {
        System.out.println("Person details");
    }
}
class Student extends Person {
@Override    // Overriding the parent class method
    void display() {
    super.display(); // Call the parent class method
       System.out.println("Student details"); // Display
student-specific details
    }
}
```

```
// Main class
class SuperDemo {
    public static void main(String[] args) {

        // Create an object of Student
        Student s1 = new Student();

        // Call the display method
        s1.display();
    }
}
```

Polymorphism

- Polymorphism means “many forms.” In object-oriented programming, it allows the same method or superclass reference to behave differently depending on the object involved.
- **Run-time polymorphism** in Java is achieved through inheritance and method overriding. The method to execute is determined at run time based on the actual object's class.
- **Compile-time polymorphism** is commonly achieved through method overloading, where multiple methods have the same name but different parameter lists.

Method Overriding

- Method overriding occurs when a subclass provides its own implementation of an inherited instance method with a compatible method signature.
- It allows the subclass to provide specialized behavior for that method.
- Syntax:

```
class Parent {
    void show() {
        System.out.println("Parent version");
    }
}
```

Method Overriding

```
class Child extends Parent {

    // Override the inherited method
    @Override
    void show() {
        System.out.println("Child version");
    }
}
```

Method Overriding Rules

- The overriding method must have the same method name and parameter types as the inherited method. Its return type must be the same or a permitted covariant reference type.

- The overriding method cannot have more restrictive access than the inherited method.

- The *@Override* annotation helps the compiler verify that the method correctly overrides an inherited method.

Method Overriding Rules

- A *final* instance method cannot be overridden.
- A *private* method is not overridden because it is not inherited as an accessible method by the subclass.
- A *static* method is hidden, not overridden.
- A method with a different parameter list is an example of method overloading, not overriding.
- The subclass can invoke the superclass implementation using *super.methodName().*

Method Overriding

```
class Employee {
    void work() {
        System.out.println("Employee performs work");
    }
}

class Teacher extends Employee {

 @Override // Override the work() method
  void work() {
        System.out.println("Teacher teaches students");
    }
}

class OverrideDemo {
    public static void main(String[] args) {

        // Parent reference refers to a Teacher object
        Employee e = new Teacher();

        e.work(); // Calls the overridden method

    }
}
```

Dynamic binding / dynamic method dispatch

- Dynamic binding, also called late binding or dynamic method dispatch, is the process in which Java determines at run time which overridden instance method to execute. The decision depends on the actual object's class, not merely the declared type of the reference variable.
- The reference type controls which members can be accessed at compile time.
- The actual object type decides which overridden implementation runs

Dynamic binding / dynamic method dispatch

- Syntax:

```
Superclass reference = new Subclass();
reference.overriddenMethod();
```

- **Superclass reference —** declares a reference variable of the superclass type.
- **new Subclass() —** creates an object of the subclass.
- **reference.overriddenMethod() —** calls the subclass implementation if the subclass overrides that instance method.

Dynamic binding / dynamic method dispatch

```
class Shape {
      double area() {      // Method to calculate area
       return 0.0;
    }
}
class Rectangle extends Shape {
    double length, breadth;
    Rectangle(double l, double b) {
        length = l;
        breadth = b;
    }
     @Override      // Override the area() method
    double area() {
        return length * breadth;
    }
}
class Circle extends Shape {
    double radius;
    Circle(double r) {
        radius = r;
    }
   @Override
    double area() {
        return Math.PI * radius * radius;
    }
}
class DispatchDemo {
    public static void main(String[] args) {
        Shape s; // Declare a superclass reference
        s = new Rectangle(4, 5); // Reference to a Rectangle object
        System.out.println(s.area());
        s = new Circle(2);      // Reference to a Circle object
        System.out.println(s.area());
    }
}
```

abstract classes and methods

- An abstract class is a class that cannot be used to create objects directly. It is designed to be inherited by other classes.

- An abstract class is a class declared using the abstract keyword

- An **abstract method** is a method declared using the abstract keyword without a method body. It ends with a semicolon.

abstract classes and methods

- An abstract class can contain two types of methods:

1. **Abstract method:** A method without a body. All the abstract methods of abstract class should be implemented in its sub classes. If not implemented in its subclasses, then that sub class must be declared as abstract.

Syntax:

```
abstract return_type method_name(parameter_list);
```

abstract classes and methods

2.Concrete method: A method with a body. The child class can use it directly without rewriting it.

Syntax:

```
return_type method_name() {  // Concrete method
// method body
}
```

- An abstract class is a class that contains one or more abstract methods.

abstract classes and methods

- An abstract class can contain instance variables, constructors, concrete methods in addition to abstract methods.
- We cannot create an object to abstract class, but we can create reference of abstract class.
- Also, you cannot declare abstract constructors or abstract static methods.

abstract classes and methods

- It is declared using the *abstract* keyword

Syntax:

```
abstract class ClassName {
abstract void methodName(); // Abstract method
void concreteMethod() { // Concrete method
System.out.println("Method implementation");
}
}
```

- *Abstract Class cannot be instantiated directly*

abstract classes and methods

```
abstract class Shape {
    abstract double area(); // abstract method
    void describe() {     // Concrete method
        System.out.println(
            "Every shape has an area calculation.");
    }
}
class Rectangle extends Shape {
    private double length, breadth;
       Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }
    @Override // Implement the abstract method
    double area() {
        return length * breadth;
    }
}
class Circle extends Shape {
    private double radius;
    Circle(double radius) {
        this.radius = radius;
    }
    @Override    // Implement the abstract method
    double area() {
        return Math.PI * radius * radius;
    }
}
```

abstract classes and methods

```
class ShapeDemo {
    public static void main(String[] args) {
            Shape s = new Rectangle(4, 3);  // Shape reference refers to a Rectangle object
        System.out.println(s.area());
        s = new Circle(2);  // Same reference refers to a Circle object
        System.out.println(s.area());
        s.describe();   // Call the concrete method of Shape
    }
}
```

Interface

- An interface in Java is like a blueprint that tells a class which methods it must provide, but it does not specify how those methods should work.

  - **What to do:** Defined by the interface.
  - **How to do it:** Defined by the implementing class.

- Focuses on what a class can do, not how it does it (implementation details are left to the class).

- Interface fields must be initialized when declared.

Interface

- Similar to an abstract class, but defines only abstract methods (no implementation).
- Acts as a contract that specifies what functionality a class must provide.
- A class can implement multiple interfaces, inheriting the required methods from each.
- Interface methods without bodies are implicitly *public* and *abstract*.

Interface

- Represents IS-A relationship.
- It is used to achieve abstraction and multiple inheritance in java.
- Variables declared in an interface are implicitly public, static, and final.
- Interfaces can contain abstract, default, and static methods. They can also contain private methods.

Defining Interface:

- An interface is declared using the *interface* keyword.
- Synatx:

```
interface InterfaceName {
    return_type method1(parameters);
    return_type method2(parameters);

    type CONSTANT_NAME = value;
}
```

Defining Interface:

Example:

```
interface Student {
    void study();
    void attendClass();

    int MAX_MARKS = 100;
}
```

Access specifiers in interfaces

- There are two common ways to declare a top-level interface.

1. **Public interface:**

- It can be accessed from other packages, subject to normal Java accessibility rules.

- A source file can have at most one top-level public type in the usual file-based arrangement.

Example:

```
public interface Animal {
    void sound();
}
```

Access specifiers in interfaces

2. Default access (package-private):

- The interface can be accessed by code within the same package.
- Code outside that package cannot directly access it.

Example:

```
interface Animal {
    void sound();
}
```

If no access modifier is written for a top-level interface, it has package-private access.

Implementing interface

- To implement an interface, Java uses the *implements* keyword.

Syntax:

```
class ClassName implements InterfaceName {
    public void methodName() {
        // Method implementation
    }
}
```

Implementing interface

- If a class extends a superclass and implements interfaces
- Syntax:

```
class ClassName extends SuperClass
        implements Interface1, Interface2 {
    // Class body
}
```

- When you implement an interface method, it must be declared as public

Implementing interface

```
interface Callback {
    void callback(int param);
}

class Client implements Callback {
    public void callback(int p) {
        System.out.println("callback called with " + p);
    }
}
```

```
class Main {
    public static void main(String[] args) {
        Client c = new Client();
        c.callback(100);
    }
}
```

Accessing Implementations through Interface References

- in Java, we can create a reference variable using an interface name instead of a class name.
- That reference can point to an object of any class that implements the interface.
- When we call a method using that reference, Java executes the correct method according to the actual object at runtime.
- This is called dynamic method dispatch and supports runtime polymorphism.

Accessing Implementations through Interface References

```
interface Callback {
    void callback(int param);
}

class Client implements Callback {
    public void callback(int p) {
        System.out.println("callback called with " + p);
    }
}
```

```
// Another implementation of Callback
class AnotherClient implements Callback {
    public void callback(int p) {
        System.out.println("Another version of callback");
        System.out.println("p squared is " + (p * p));
    }
}
```

Accessing Implementations through Interface References

```
class TestIface2 {
    public static void main(String[] args) {
        Callback c = new Client();

        AnotherClient ob = new AnotherClient();

        c.callback(42);

        c = ob;  // c now refers to AnotherClient object

        c.callback(42);
    }
}
```

extending interfaces

- In Java, one interface can inherit another interface using the `extends` keyword.
- When an interface inherits another interface, it gets the method declarations of the parent interface.
- A class that implements the child interface must implement all inherited abstract methods, unless the class is declared `abstract`.

extending interfaces

```
interface Student {
    void study();
    void attendClass();
}

interface Teacher extends Student {
    void teach();
}

class SchoolTeacher implements Teacher {

    public void study() {
        System.out.println("Student studies.");
    }

    public void attendClass() {
        System.out.println("Student attends class.");
    }

    public void teach() {
        System.out.println("Teacher teaches students.");
    }
}

class SchoolDemo {
    public static void main(String[] args) {
        SchoolTeacher obj = new SchoolTeacher();

        obj.study();
        obj.attendClass();
        obj.teach();
    }
}
```

Interfaces vs. Abstract classes

Feature	Abstract Class	Interface
Main purpose	Provides a common base with partial implementation.	Defines a contract that implementing classes must follow.
Declaration	abstract class A { }	interface I { }
Methods	Can contain abstract and concrete methods.	Methods are implicitly public and abstract in the traditional model.
Variables / State	Can have instance variables to store object-specific data.	Fields are implicitly public, static, and final constants.
Constructors	Can declare constructors.	Cannot declare constructors.
Inheritance by a class	A class can extend only one class.	A class can implement multiple interfaces.
Inheritance keyword	Uses extends.	Uses implements for classes; interfaces use extends to inherit other interfaces.

Interfaces vs. Abstract classes

Feature	Abstract Class	Interface
Object creation	Cannot be instantiated directly.	Cannot be instantiated directly.
Subclass / Implementer responsibility	A concrete subclass must implement inherited abstract methods that remain unimplemented.	A concrete implementing class must implement the required abstract methods.
Best suited for	Related classes that share state and behavior.	Common capabilities shared by potentially unrelated classes.
Access modifiers	Methods can use different access modifiers, subject to overriding rules.	Traditional abstract interface methods are implicitly public.
Example	abstract class Shape { abstract double area(); }	interface Printable { void print(); }

Access Specifiers In Java

- Access specifiers (also called access modifiers) control the visibility and accessibility of classes, methods, constructors, and variables in Java.

Access Specifier	Within Same Class	Same Package	Subclass in Different Package	Other Packages (Non-subclass)
private	Yes	No	No	No
default (no keyword)	Yes	Yes	No*	No
protected	Yes	Yes	Yes*	No
public	Yes	Yes	Yes	Yes

